



UNIVERSIDAD TECNOLÓGICA DEL PERÚ

Facultad de Ingeniería de Sistemas y Electrónica

SISTEMAS OPERATIVOS

Algoritmos Criptográficos en Sistemas Operativos modernos

Estudiante	Max Anderson Benites Corazón
Ciclo	V Ciclo
Año	2026

2. INVESTIGACIÓN Y EXPLICACIÓN

AES — Cifrado simétrico

¿Qué es? AES (Advanced Encryption Standard) es un cifrador de bloques simétrico: la misma clave secreta cifra y descifra la información. Trabaja con bloques de 128 bits y admite claves de 128, 192 o 256 bits. Su operación combina varias rondas de sustitución, permutación y mezcla para transformar los datos en texto cifrado.

Uso en el sistema operativo. Se emplea para proteger datos en reposo, por ejemplo en el cifrado de discos, volúmenes y archivos. BitLocker utiliza AES en modos orientados al almacenamiento, mientras que Linux puede aplicar AES mediante dm-crypt/LUKS. El sistema conserva la clave protegida y cifra o descifra sectores de forma transparente durante las operaciones de entrada/salida.

Punto clave: AES es rápido y adecuado para grandes volúmenes de datos, pero la seguridad depende de proteger la clave y utilizar correctamente el modo de operación y los vectores de inicialización.

RSA — Criptografía asimétrica

¿Qué es? RSA utiliza un par de claves relacionadas: una pública que puede compartirse y una privada que debe permanecer secreta. Su seguridad se basa en la dificultad práctica de factorizar un número compuesto muy grande. Puede cifrar cantidades pequeñas de información y producir firmas digitales verificables con la clave pública.

Uso en el sistema operativo. Los sistemas operativos pueden verificar con RSA la firma de módulos del kernel antes de cargarlos, evitando la ejecución de componentes no autorizados. En SSH, RSA se utiliza principalmente para autenticar al servidor o al usuario mediante firmas durante el handshake; el secreto de sesión normalmente se negocia con algoritmos específicos de intercambio de claves y luego se cifra con un algoritmo simétrico.

Punto clave: RSA no se usa para cifrar archivos completos por su costo y límite de tamaño; se combina con cifrado simétrico o se usa para firmas.

SHA-256 — Función hash

¿Qué es? SHA-256 recibe datos de cualquier longitud y genera un resumen fijo de 256 bits. Es una función unidireccional: no existe una operación de descifrado. Un cambio mínimo en la entrada produce un hash muy diferente, lo que permite detectar modificaciones.

Uso en el sistema operativo. Se usa para comparar la integridad de archivos, paquetes, imágenes del sistema y registros. También puede formar parte de esquemas de contraseñas con sal y múltiples rondas. No debe almacenarse una contraseña aplicando SHA-256 una sola vez; para ese fin se prefieren funciones de derivación lentas como Argon2, scrypt o PBKDF2.

Punto clave: SHA-256 verifica integridad, pero por sí solo no demuestra quién creó los datos; para autenticidad debe combinarse con una firma digital o un HMAC.

3. IMPLEMENTACIÓN EN JAVA

Clase ejecutable con AES-CBC, RSA-OAEP de 2048 bits y SHA-256. Requiere Java 8 o superior.

```
import java.nio.ByteBuffer;
import java.nio.charset.StandardCharsets;
import java.security.KeyPair;
import java.security.KeyPairGenerator;
import java.security.MessageDigest;
import java.security.PrivateKey;
import java.security.PublicKey;
import java.security.SecureRandom;
import java.security.spec.MGF1ParameterSpec;
import java.util.Base64;

import javax.crypto.Cipher;
import javax.crypto.KeyGenerator;
import javax.crypto.SecretKey;
import javax.crypto.spec.IvParameterSpec;
import javax.crypto.spec.OAEPParameterSpec;
import javax.crypto.spec.PSource;

public class LaboratorioCriptografia {
    private static final String AES_TRANSFORMATION = "AES/CBC/PKCS5Padding";
    private static final String RSA_TRANSFORMATION =
        "RSA/ECB/OAEPWithSHA-256AndMGF1Padding";
    private static final SecureRandom RANDOM = new SecureRandom();

    public static String cifrarAES(String texto, SecretKey clave) throws Exception {
        byte[] iv = new byte[16];
        RANDOM.nextBytes(iv);

        Cipher cipher = Cipher.getInstance(AES_TRANSFORMATION);
        cipher.init(Cipher.ENCRYPT_MODE, clave, new IvParameterSpec(iv));
        byte[] cifrado = cipher.doFinal(texto.getBytes(StandardCharsets.UTF_8));

        ByteBuffer resultado = ByteBuffer.allocate(iv.length + cifrado.length);
        resultado.put(iv);
        resultado.put(cifrado);
        return Base64.getEncoder().encodeToString(resultado.array());
    }

    public static String descifrarAES(String datosBase64, SecretKey clave)
        throws Exception {
        byte[] datos = Base64.getDecoder().decode(datosBase64);
        ByteBuffer buffer = ByteBuffer.wrap(datos);
        byte[] iv = new byte[16];
        buffer.get(iv);
        byte[] cifrado = new byte[buffer.remaining()];
        buffer.get(cifrado);

        Cipher cipher = Cipher.getInstance(AES_TRANSFORMATION);
        cipher.init(Cipher.DECRYPT_MODE, clave, new IvParameterSpec(iv));
        byte[] textoPlano = cipher.doFinal(cifrado);
        return new String(textoPlano, StandardCharsets.UTF_8);
    }

    private static OAEPParameterSpec parametrosOAEP() {
        return new OAEPParameterSpec(
            "SHA-256",
            "MGF1",
            MGF1ParameterSpec.SHA256,
            PSource.PSpecified.DEFAULT);
    }

    public static String cifrarRSA(String texto, PublicKey clavePublica)
        throws Exception {
        Cipher cipher = Cipher.getInstance(RSA_TRANSFORMATION);
        cipher.init(Cipher.ENCRYPT_MODE, clavePublica, parametrosOAEP());
        byte[] cifrado = cipher.doFinal(texto.getBytes(StandardCharsets.UTF_8));
        return Base64.getEncoder().encodeToString(cifrado);
    }

    public static String descifrarRSA(String datosBase64, PrivateKey clavePrivada)
        throws Exception {
    
```

```

        Cipher cipher = Cipher.getInstance(RSA_TRANSFORMATION);
        cipher.init(Cipher.DECRYPT_MODE, clavePrivada, parametrosOAEP());
        byte[] cifrado = Base64.getDecoder().decode(datosBase64);
        byte[] textoPlano = cipher.doFinal(cifrado);
        return new String(textoPlano, StandardCharsets.UTF_8);
    }

    public static String calcularSHA256(String texto) throws Exception {
        MessageDigest digest = MessageDigest.getInstance("SHA-256");
        byte[] hash = digest.digest(texto.getBytes(StandardCharsets.UTF_8));
        StringBuilder hexadecimal = new StringBuilder(hash.length * 2);
        for (byte valor : hash) {
            hexadecimal.append(String.format("%02x", valor & 0xff));
        }
        return hexadecimal.toString();
    }

    public static void main(String[] args) throws Exception {
        String texto = "Sistemas Operativos UTP - 2026";

        KeyGenerator generadorAES = KeyGenerator.getInstance("AES");
        generadorAES.init(128);
        SecretKey claveAES = generadorAES.generateKey();
        String cifradoAES = cifrarAES(texto, claveAES);
        String descifradoAES = descifrarAES(cifradoAES, claveAES);

        KeyPairGenerator generadorRSA = KeyPairGenerator.getInstance("RSA");
        generadorRSA.initialize(2048);
        KeyPair parRSA = generadorRSA.generateKeyPair();
        String cifradoRSA = cifrarRSA(texto, parRSA.getPublic());
        String descifradoRSA = descifrarRSA(cifradoRSA, parRSA.getPrivate());

        String hashSHA256 = calcularSHA256(texto);

        System.out.println("=== LABORATORIO DE CRIPTOGRAFIA ===");
        System.out.println("Texto original : " + texto);
        System.out.println("\n[AES]");
        System.out.println("Cifrado Base64 : " + cifradoAES);
        System.out.println("Descifrado      : " + descifradoAES);
        System.out.println("\n[RSA]");
        System.out.println("Cifrado Base64 : " + cifradoRSA);
        System.out.println("Descifrado      : " + descifradoRSA);
        System.out.println("\n[SHA-256]");
        System.out.println("Hash hexadecimal: " + hashSHA256);
    }
}

```